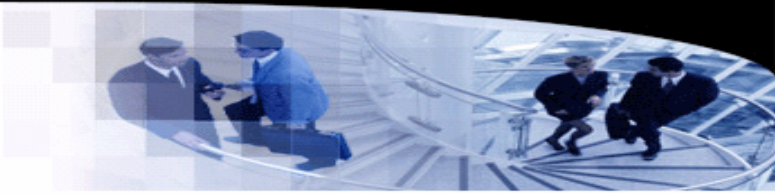
武漢大學
WUHAN UNIVERSITY

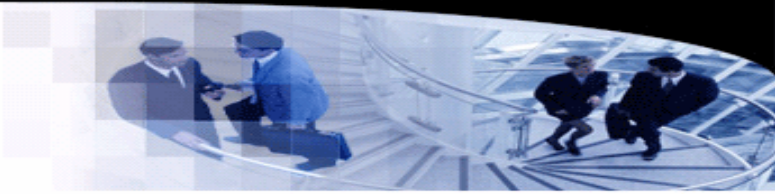


Programming Languages and Compilers

Lecture 6. Context-free Languages

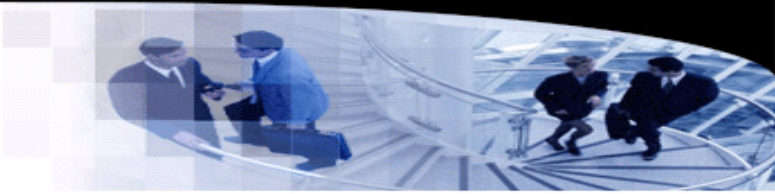
袁梦霆 (YUAN Mengting)
ymt@whu.edu.cn
School of Computer Science, Wuhan University

Introduction



- Context-free grammar (CFG)
  - Grammars in which all productions are of the form $A \to \alpha$, where $A \in V_N$.
- Context-free languages (CFL)
  - A language L is context-free, if there is a context-free grammar G such that $L = \mathcal{L}(G)$.
  - Most programming languages are context-free.
- Pushdown automata (PDA)
  - A pushdown automaton is a state machine with a stack.
  - Pushdown automata can recognize (parse) context-free languages.
  - It implies that we can use stacks to simulate pushdown automata, so as to parse programming languages.

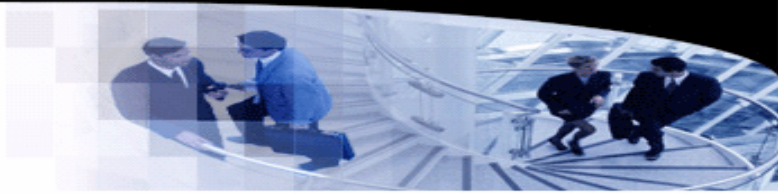
Pushdown Automata



- Example
  - Design an algorithm, in which a state variable and a stack are allowed, to recognize the language $\{0^n 1^n \mid n > 0\}$.
  - Algorithm:

```
s = 0;
while not eof do
  if s = 0 then
    if a = '0' then push('0'); endif
    if a = '1' then
      s = 1;
      pop(); // Reject if the stack is empty.
    endif
    if a != '0' or a != '1' then reject(); endif
  else
    if a = '1' then
      pop(); // Reject if the stack is empty.
    else
      reject();
    endif
  endif
endwhile
if s = 1 and emptystack() then accept(); else reject(); endif
```

Pushdown Automata



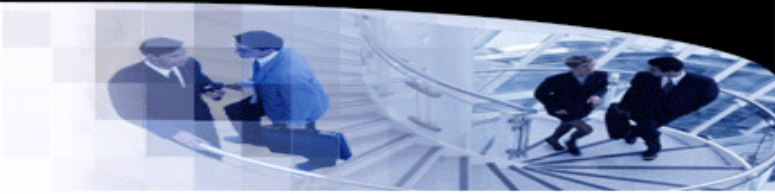
- Example
  - Design an algorithm, in which a state variable and a stack are allowed, to recognize the language $\{0^n 1^n \mid n > 0\}$.
  - Algorithm:

```
s = 0;
while not eof do
  if s = 0 then
    if a = '0' then push('0'); endif
    if a = '1' then
      s = 1;
      pop(); // Reject if the stack is empty.
    endif
    if a != '0' or a != '1' then reject(); endif
  else
    if a = '1' then
      pop(); // Reject if the stack is empty.
    else
      reject();
    endif
  endif
endwhile
if s = 1 and emptystack() then accept(); else reject(); endif
```

Stage 0: Push "0"s to the stack.

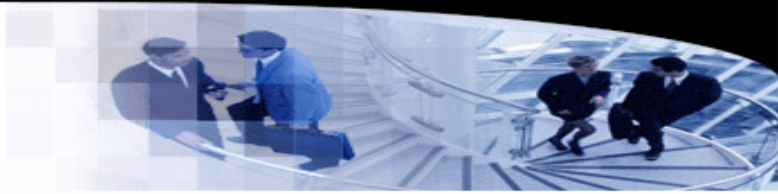
Stage 1: Pop a "0" out of the stack when an "1' is read.

Pushdown Automata

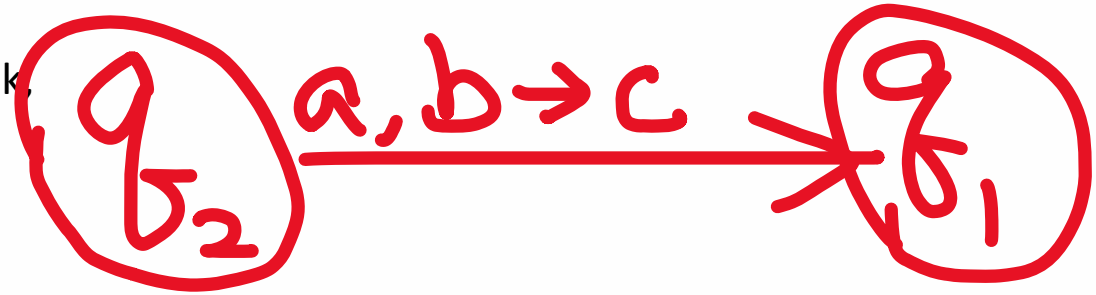


- Syntax
  - A pushdown automaton is a tuple $M = (Q, \Sigma, \Gamma, q_0, \delta, F)$, where
    - Q is the set of states,
    - Σ is the alphabet (of the input),
    - Γ is the set of symbols that occurs in the stack,
    - q_0 is the start state,
    - $\delta: Q \times \Sigma_\epsilon \times \Gamma_\epsilon \to \mathcal{P}(Q \times \Gamma_\epsilon)$,
    - and $F \subseteq Q$ is the set of accepting states.
  - $\Sigma_\epsilon = \Sigma \cup \{\epsilon\}$ and $\Gamma_\epsilon = \Gamma \cup \{\epsilon\}$.
  - Pushdown automata are **nondeterministic**.
  - The transition function δ tells how to change the top of the stack based on the current top of the stack, the state, and the input symbol.
    - $(q_1, c) \in \delta(q_2, a, b)$ means that if the current state is q_2 and the input is a and the top of the stack is b, then the state is updated to q_1 and the top of the stack is updated to c.

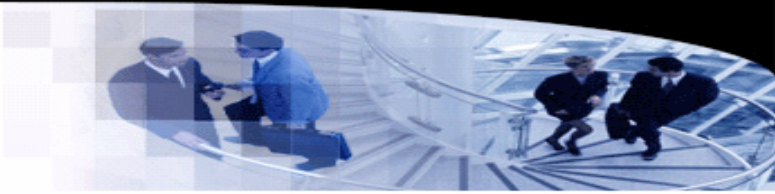
Pushdown Automata



- Syntax
  - A pushdown automaton is a tuple $M = (Q, \Sigma, \Gamma, q_0, \delta, F)$, where
    - Q is the set of states,
    - Σ is the alphabet (of the input),
    - Γ is the set of symbols that occurs in the stack,
    - q_0 is the start state,
    - $\delta: Q \times \Sigma_\epsilon \times \Gamma_\epsilon \to \mathcal{P}(Q \times \Gamma_\epsilon)$,
    - and $F \subseteq Q$ is the set of accepting states.
  - $\Sigma_\epsilon = \Sigma \cup \{\epsilon\}$ and $\Gamma_\epsilon = \Gamma \cup \{\epsilon\}$.
  - Pushdown automata are **nondeterministic**.
  - The transition function δ tells how to change the top of the stack based on the current top of the stack, the state, and the input symbol.
    - $(q_1, c) \in \delta(q_2, a, b)$ means that if the current state is q_2 and the input is a and the top of the stack is b, then the state is updated to q_1 and the top of the stack is updated to c.



Pushdown Automata

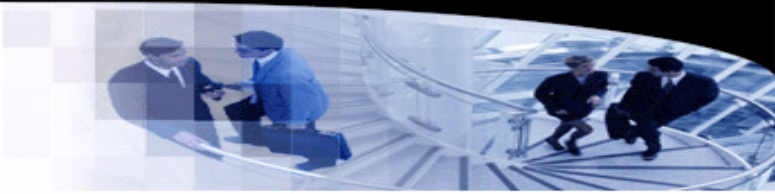


- Syntax
  - A pushdown automaton is a tuple $M = (Q, \Sigma, \Gamma, q_0, \delta, F)$, where
    - Q is the set of states,
    - Σ is the alphabet (of the input),
    - Γ is the set of symbols that occurs in the stack,
    - q_0 is the start state,
    - $\delta: Q \times \Sigma_\epsilon \times \Gamma_\epsilon \to \mathcal{P}(Q \times \Gamma_\epsilon)$,
    - and $F \subseteq Q$ is the set of accepting states.
  - $\Sigma_\epsilon = \Sigma \cup \{\epsilon\}$ and $\Gamma_\epsilon = \Gamma \cup \{\epsilon\}$.
  - Pushdown automata are **nondeterministic**.
  - The transition function δ tells how to change the top of the stack based on the current top of the stack, the state, and the input symbol.
    - $(q_1, c) \in \delta(q_2, a, b)$ means that if the current state is q_2 and the input is a and the top of the stack is b, then the state is updated to q_1 and the top of the stack is updated to c.

Push c to the stack.

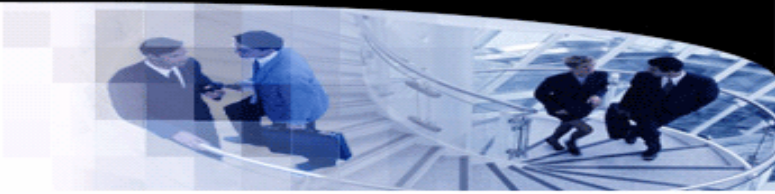
Pop b out of the stack.

Pushdown Automata



- Semantics
  - The automaton M starts from the start state.
  - M reads input symbols one by one.
  - M calculates next state(s) by its transition function, and forks multiple copies for all nondeterministic target states.
  - Each copy of M has its **own** input and stack. Note that NFA copies share the same input.
  - If the input of a copy ends, the copy accepts or rejects the input according to the current state.
  - M accepts the input if at least one copy accepts the input. The input is rejected otherwise.

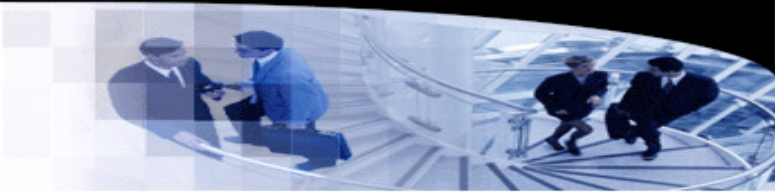
Pushdown Automata



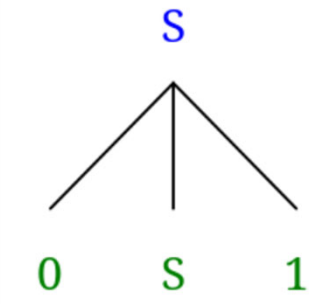
- A top-down parser based on PDA
  - Given a grammar G and an input α, a **parser** checks whether $\alpha \in \mathcal{L}(G)$.
  - One possible strategy may try to find a sequence of derivations such that $S \Rightarrow \beta_1 \Rightarrow \beta_2 \Rightarrow \cdots \Rightarrow \alpha$.
  - Since the finding of the sequence implies the syntax tree is drawn from the top to the bottom, this strategy is called top-down parsing.
- Example
  - Let $G_1[S]: S \rightarrow 0S1|01$, and the $\alpha = 0011$.
  - We have the derivations:
    - S

S

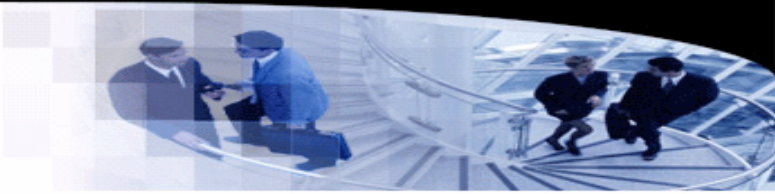
Pushdown Automata



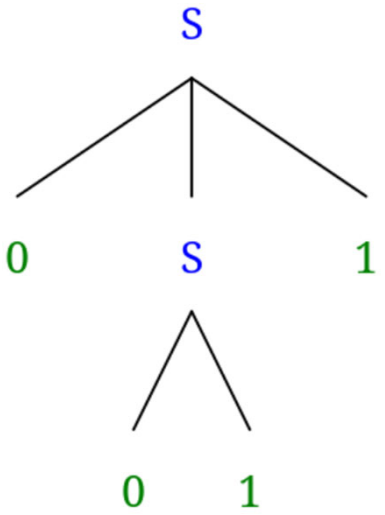
- A top-down parser based on PDA
  - Given a grammar G and an input α, a **parser** checks whether $\alpha \in \mathcal{L}(G)$.
  - One possible strategy may try to find a sequence of derivations such that $S \Rightarrow \beta_1 \Rightarrow \beta_2 \Rightarrow \cdots \Rightarrow \alpha$.
  - Since the finding of the sequence implies the syntax tree is drawn from the top to the bottom, this strategy is called top-down parsing.
- Example
  - Let $G_1[S]: S \rightarrow 0S1|01$, and the $\alpha = 0011$.
  - We have the derivations:
    - S
    - $\Rightarrow 0S1$



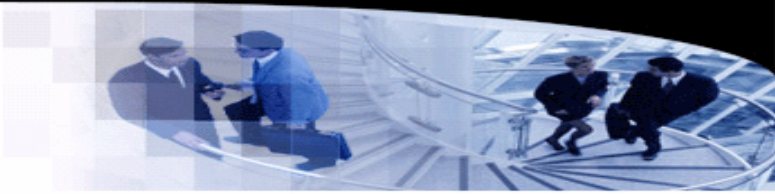
Pushdown Automata



- A top-down parser based on PDA
  - Given a grammar G and an input α, a **parser** checks whether $\alpha \in \mathcal{L}(G)$.
  - One possible strategy may try to find a sequence of derivations such that $S \Rightarrow \beta_1 \Rightarrow \beta_2 \Rightarrow \cdots \Rightarrow \alpha$.
  - Since the finding of the sequence implies the syntax tree is drawn from the top to the bottom, this strategy is called top-down parsing.
- Example
  - Let $G_1[S]: S \rightarrow 0S1|01$, and the $\alpha = 0011$.
  - We have the derivations:
    - S
    - $\Rightarrow 0S1$
    - $\Rightarrow 0011$



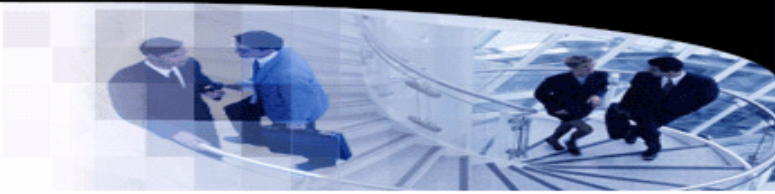
Pushdown Automata



- A top-down parser based on PDA
  - Idea of the parser
    - The stack stores the intermediates result of derivations.
    - For each step of derivation, the left-most nonterminal symbol is chosen from the stack.
    - The nondeterministic nature of PDA is used to deal with multiple productions for the same nonterminal symbol. If $A \rightarrow \alpha_1 | \alpha_2 | \ldots | \alpha_n$, the PDA forks n copies, each of which replaces A with an α_i.

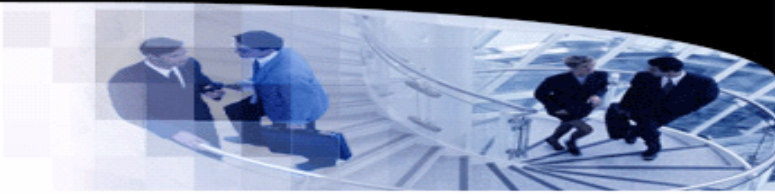
Why is "0" placed on the top of the stack?

Pushdown Automata

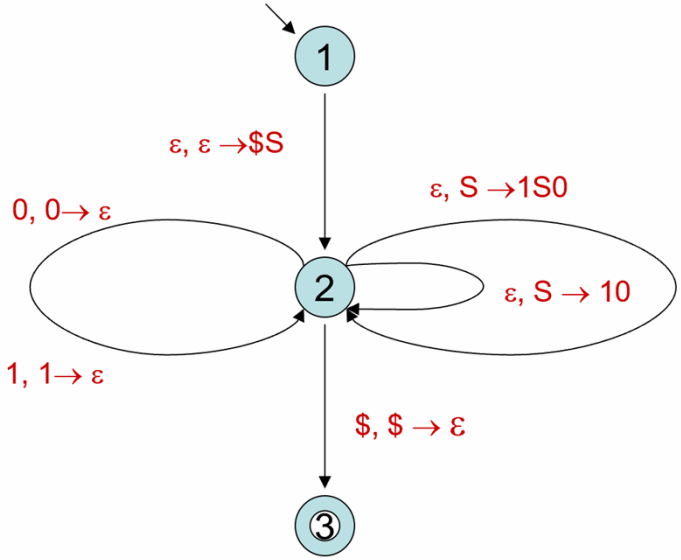


- A top-down parser based on PDA
  - Algorithm
    - Push "$\$S$" into the stack.
    - If the top of the stack is a nonterminal A, replace A with the right-hand-side of the production nondeterministicly.
    - If the top of the stack is a terminal a and
      - the input symbol is a as well, pop a from the stack.
      - the input symbol is not a, the copy rejects the input.
    - If the input symbol is $ (which implies the end of the input) and the top of the stack is $, the input is accepted.

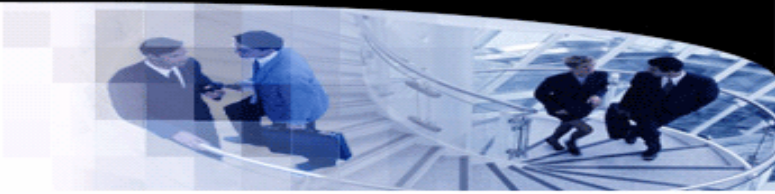
Pushdown Automata



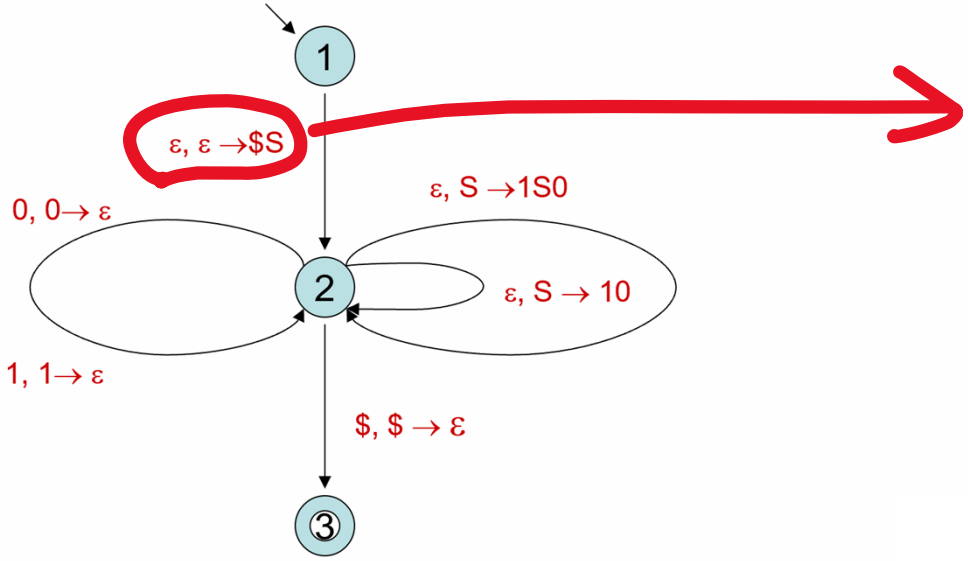
- A top-down parser based on PDA
  - Given a grammar $G = (V_N, V_T, P, S)$, the parser PDA is constructed as follows.
    - Generate two transitions to push "$" and "S".
    - For each production $A \rightarrow \alpha$, generate transitions to pop "A" and push "α".
    - For each terminal a, generate a transition to pop "a" out of the stack.
    - Generate a transition to "guess" whether the parser has generated the input string.
  - Example
    - $G_1[S]: S \rightarrow 0S1|01$



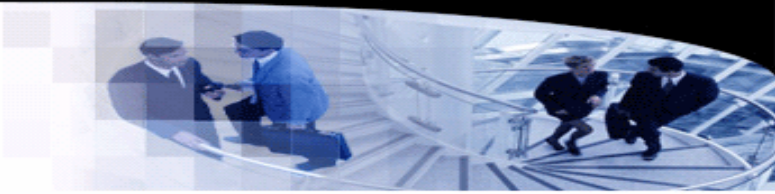
Pushdown Automata



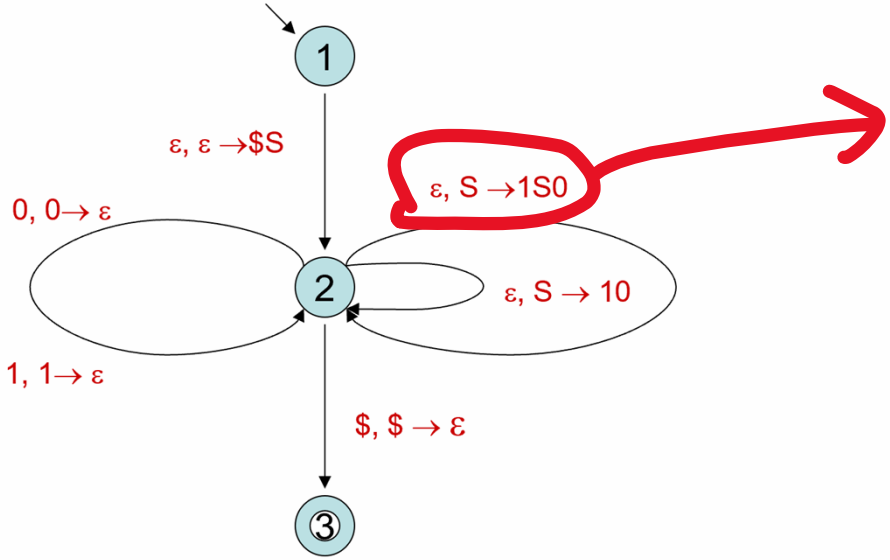
- A top-down parser based on PDA
  - Given a grammar $G = (V_N, V_T, P, S)$, the parser PDA is constructed as follows.
    - Generate two transitions to push "$" and "S".
    - For each production $A \rightarrow \alpha$, generate transitions to pop "A" and push "α".
    - For each terminal a, generate a transition to pop "a" out of the stack.
    - Generate a transition to "guess" whether the parser has generated the input string.
  - Example
    - $G_1[S]: S \rightarrow 0S1|01$

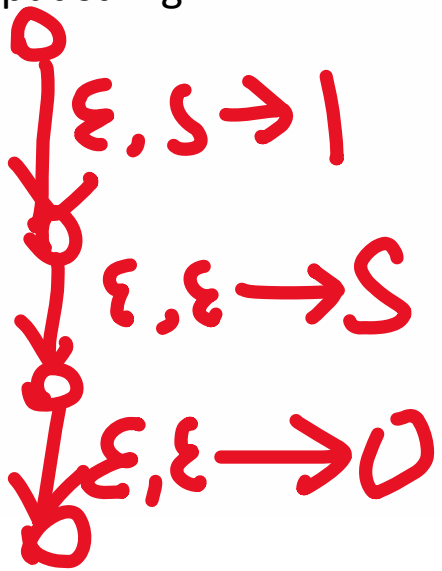


Pushdown Automata



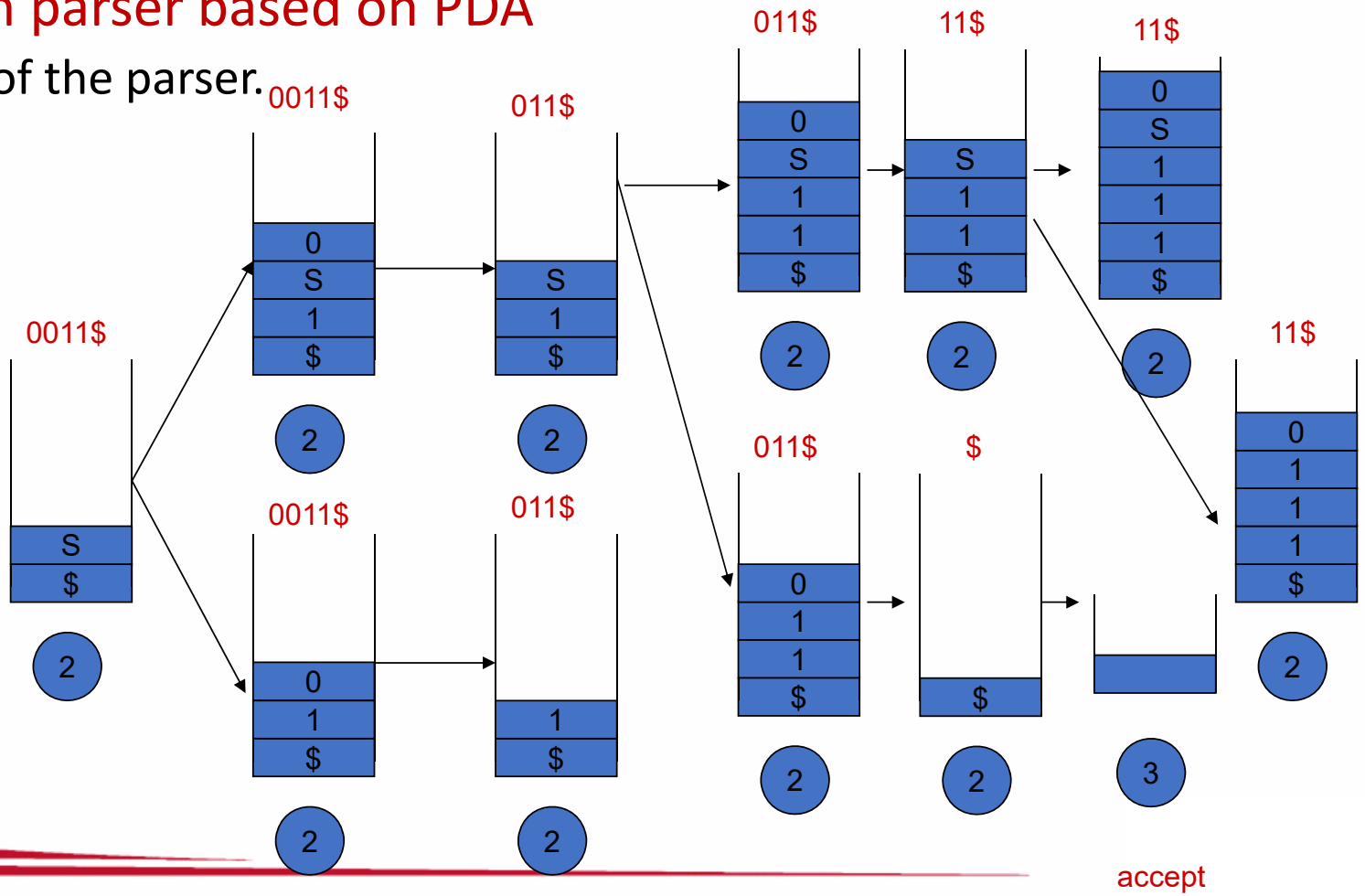
- A top-down parser based on PDA
  - Given a grammar $G = (V_N, V_T, P, S)$, the parser PDA is constructed as follows.
    - Generate two transitions to push "$" and "S".
    - For each production $A \rightarrow \alpha$, generate transitions to pop "A" and push "α".
    - For each terminal a, generate a transition to pop "a" out of the stack.
    - Generate a transition to "guess" whether the parser has generated the input string.
  - Example
    - $G_1[S]: S \rightarrow 0S1|01$



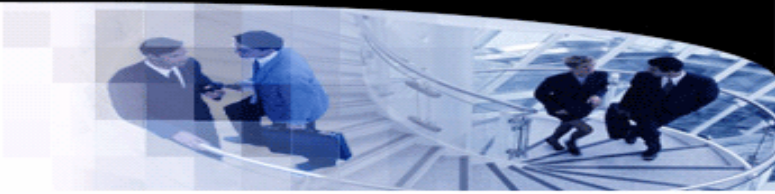


Pushdown Automata

- A top-down parser based on PDA
  - Running of the parser.



Pushdown Automata



- The whole story
  - Given a context-free grammar, we can generate a pushdown automaton automatically to parse the language.
  - By simulating this PDA, we obtain a parser for the language.
  - However, this method has a serious performance issue caused by the nondeterministic mechanism.
  - Later, we will introduce deterministic context-free languages, which can be parsed by deterministic PDA efficiently.
  - Spoiler: LL(k) and LR(k) languages.